

SMART INDIA HACKATHON 2026

# **Project Report**

**Project Title:** JAGRUK SWADESH AI-Powered BIS Compliance & Conversational Knowledge Assistant

**Team Details:**

- **Team Name:** Team Jagruk Swadesh
- **Problem Statement ID:** SIH26107
- **Problem Statement Title:** AI-powered Intelligent Assistant for Indian Standards and BIS Services for Industries and Consumers
- **Theme:** Smart Automation

**Submitted by:**

- Om Kumar Roy (Team Leader)
- Videesh Sharma (Team Member)
- Om Jee (Team Member)
- Sonu Kumar (Team Member)
- Shruti Yadav (Team Member)
- Saurabh devliyal (Team Member)

**Institution:** Uttaranchal University (UIT), Prem nagar , Dehradun, Uttarakhand

# **CERTIFICATE OF AUTHENTICITY**

This is to certify that the project report entitled "JAGRUK SWADESH: AI-Powered BIS Compliance & Conversational Knowledge Assistant" is a bona fide record of the original software development work carried out by Team Jagruk Swadesh for the Smart India Hackathon 2026.

The system implementation, data pipelines, AI architecture, and source code described in this document were conceptualized and developed by the team members from UIT Uttarakhand to address the specific requirements of Problem Statement [Insert PS ID] provided by [Insert Ministry/Organization].

We certify that this work represents our actual implementation developed during the SIH 2026 phases and has not been copied from pre-existing unoriginal sources or submitted elsewhere for any other hackathon or degree without proper disclosure.

# **DECLARATION FOR SIH 2026**

We, the members of Team Jagruk Swadesh, hereby declare that the prototype and report submitted for the Smart India Hackathon 2026 are entirely our original collaborative work.

We affirm that the architecture described—including the RAG implementation, the LangGraph-based "Jagruk Brain," and the automated BIS data extraction pipeline—accurately reflects the system we have deployed for evaluation. All third-party tools, APIs (including Groq and Whisper), and open-source libraries (such as React, FastAPI, and Qdrant) utilized to build this platform have been properly acknowledged in the technical stack documentation.

We agree to abide by the rules and regulations of the Smart India Hackathon 2026 and confirm that our solution aims to fulfill the vision of making Indian Standards simple and accessible.

**ABSTRACT**

The Bureau of Indian Standards (BIS) provides critical regulations, certifications, and quality control orders necessary for manufacturing, safety, and compliance in India. However, accessing and comprehending these extensive, highly technical documents poses a significant challenge for ordinary citizens, small business owners, and non-technical users.

**Jagruk Swadesh** is an AI-powered conversational platform designed to solve this problem by simplifying access to BIS standards. Serving as a bridge between complex regulatory frameworks and end-users, the platform embodies the vision: *"Indian Standards Made Simple."*

The system utilizes a robust microservices architecture combining a modern React web application with a Retrieval-Augmented Generation (RAG) pipeline. To ensure accuracy and prevent hallucination, the system relies on an automated data collection pipeline that ingests, structures, and embeds official BIS data into a Qdrant vector database. The "Jagruk Brain," orchestrated via a state-based pipeline, handles multimodal inputs (text and voice), processes speech using Whisper STT, performs intent extraction and query understanding, retrieves relevant evidence, and generates grounded, multilingual responses using Groq-hosted LLMs. By providing natural language voice and text interactions, Jagruk Swadesh democratizes access to regulatory knowledge, making compliance information highly accessible, reliable, and user-friendly.

## TABLE OF CONTENTS

### 1. Introduction

2. Existing System and Problem Analysis
3. System Overview
4. Technology Stack
5. BIS Data Collection and Knowledge Preparation
6. RAG Knowledge Base
7. Jagruk Brain AI Pipeline
8. Query Understanding and AI Reasoning
9. Voice and Multilingual Interaction
10. Frontend Application
11. Backend Architecture
12. API Design and Data Flow
13. Database and Data Management
14. Security
15. Testing and Validation
16. Deployment and Setup
17. Performance, Scalability and Reliability
18. Limitations and Challenges
19. Future Scope
20. Impact and Benefits
21. Conclusion
- Appendices
- WEBSITE IMAGES

## **LIST OF FIGURES**

- **Figure 4.1:** Jagruk Swadesh Technology Stack
- **Figure 5.1:** BIS Data Collection & Document Extraction Pipeline
- **Figure 7.1:** Jagruk Brain Pipeline
- **Figure 10.1:** Jagruk Swadesh Frontend User Workflow

- **Figure 11.1:** Jagruk Swadesh Backend Workflow

## **LIST OF TABLES**

- **Table 2.1:** Traditional BIS Information Access vs Jagruk Swadesh
- **Table 3.1:** System Components and Responsibilities
- **Table 12.1:** Core API Endpoints
- **Table 15.1:** Recommended Test Cases

## **LIST OF ABBREVIATIONS**

- **AI:** Artificial Intelligence
- **API:** Application Programming Interface
- **BIS:** Bureau of Indian Standards
- **CORS:** Cross-Origin Resource Sharing
- **IS:** Indian Standard
- **JSON:** JavaScript Object Notation
- **JWT:** JSON Web Token
- **LLM:** Large Language Model
- **QCO:** Quality Control Order
- **RAG:** Retrieval-Augmented Generation
- **REST:** Representational State Transfer
- **SPA:** Single Page Application
- **STT:** Speech-to-Text
- **TTS:** Text-to-Speech
- **UI/UX:** User Interface / User Experience

## **CHAPTER 1 — INTRODUCTION**

### **1.1 Background**

The Bureau of Indian Standards (BIS) is the National Standards Body of India, responsible for the harmonious development of the activities of standardization, marking, and quality certification of goods. Indian Standards (IS) ensure product quality, regulatory compliance, and consumer safety. However, the sheer volume and technical density of these documents make them difficult to navigate for laymen.

## 1.2 Problem Statement

Users face significant hurdles when trying to access and understand BIS standards. The primary challenges include:

- **Information Fragmentation:** Data is scattered across multiple PDFs, gazettes, and manuals.
- **Technical Terminology:** Standards are written in dense legal and technical language.
- **Accessibility:** Finding the correct standard for a specific product is difficult for non-technical users.
- **Language Barriers:** Most official standards are in English, alienating regional users.

## 1.3 Motivation

The project was developed to democratize regulatory knowledge. Empowering citizens and businesses with easy-to-understand compliance data fosters better manufacturing practices and consumer awareness.

## 1.4 Proposed Solution

Jagruk Swadesh is a full-stack, AI-driven conversational assistant that translates complex BIS documents into easily understandable, evidence-backed answers using a dedicated Retrieval-Augmented Generation (RAG) architecture.

## 1.5 Objectives

- To automate the extraction and structuring of BIS knowledge.
- To build an evidence-grounded AI assistant that prevents hallucination.
- To provide seamless text and voice interfaces for querying standards.
- To support multilingual interactions for wider accessibility.

## 1.6 Scope

The scope includes citizen awareness, standards discovery, product information, and conversational assistance. It features voice interaction, multilingual understanding, and direct BIS knowledge retrieval.

### **1.7 Target Users**

- **Consumers/Citizens:** Seeking safety standards for daily products.
- **Manufacturers:** Needing compliance requirements for certification.
- **Students/Researchers:** Studying industrial standards.

### **1.8 Major Features**

- Conversational AI chatbot with context memory.
- Voice input/output using Whisper and Edge TTS.
- Strict evidence-grounded RAG to prevent LLM hallucinations.
- Automated semantic vector retrieval.

### **1.9 Project Contributions**

Technically, the project contributes a specialized domain-specific RAG pipeline capable of handling Indian bureaucratic data, wrapped in a scalable microservices architecture. Practically, it transforms how citizens interact with government standards.

## **CHAPTER 2 — EXISTING SYSTEM AND PROBLEM ANALYSIS**

### **2.1 Existing BIS Information Access Process**



Currently, users must manually navigate official portals, use exact keyword searches, download massive PDF files, and manually read through hundreds of pages to find a single compliance clause.

**2.2 Limitations of Conventional Information Retrieval**

Traditional keyword search fails when users do not know the exact "IS Number" or official nomenclature of a product.

**2.6 Limitations of Generic Conversational AI**

Generic LLMs (like standard ChatGPT) hallucinate regulatory information. For compliance, hallucination is legally and practically unacceptable.

**2.7 Need for Evidence-Grounded AI**

Jagruk Swadesh restricts the AI's reasoning strictly to the retrieved context from the vector database, ensuring high fidelity and factual accuracy.

**2.9 Proposed Improvement**

**Table 2.1: Traditional BIS Information Access vs Jagruk Swadesh**

| Feature       | Traditional Access      | Jagruk Swadesh                  |
|---------------|-------------------------|---------------------------------|
| Search Method | Keyword/IS Number match | Semantic/Natural Language       |
| Data Format   | Massive PDFs            | Synthesized, bite-sized answers |
| Language      | Primarily English       | Multilingual support            |
| Interface     | Complex portals         | Intuitive Text/Voice Chat       |
| Accuracy      | High (but hard to find) | High (Evidence-grounded RAG)    |

**CHAPTER 3 — SYSTEM OVERVIEW**

**3.1 Product Vision**

To be the ultimate, accessible gateway for Indian compliance information.

3.2 Product Mission

To build "Indian Standards Made Simple" through advanced, user-centric AI.

3.4 High-Level Architecture

The system operates on a microservices-inspired architecture: User → React Frontend → Express API Gateway → FastAPI AI Service → Jagruk Brain (LangGraph) → RAG (Qdrant) → AI Reasoning (Groq) → Response → Frontend.

3.8 Major System Components

Table 3.1: System Components

| Component   | Responsibility                             | Technology                    |
|-------------|--------------------------------------------|-------------------------------|
| Frontend UI | User interaction, audio capture, rendering | React.js, Vite, Tailwind      |
| API Gateway | Auth, routing, rate limiting               | Node.js, Express, MongoDB     |
| AI Service  | Orchestrating RAG, TTS, STT                | Python, FastAPI               |
| Vector DB   | Semantic search of BIS chunks              | Qdrant, sentence-transformers |
| LLM Engine  | Intent extraction, generation              | Groq API                      |

CHAPTER 4 — TECHNOLOGY STACK

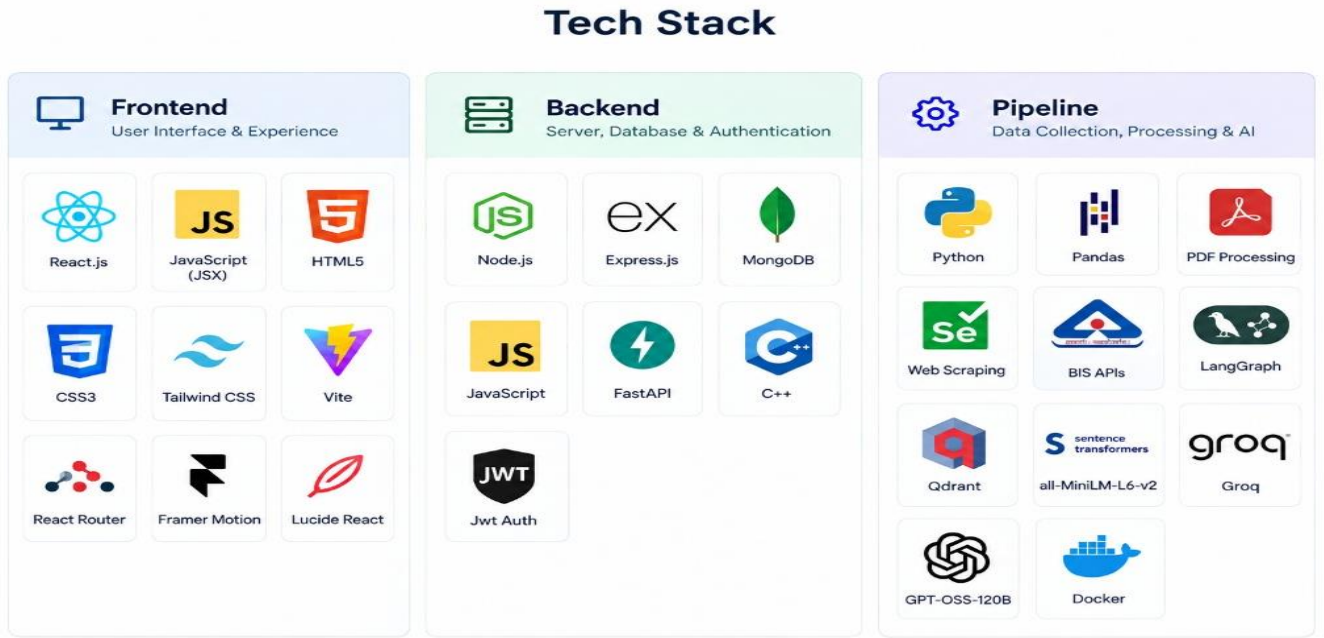


Figure 4.1: Jagruk Swadesh Technology Stack

The platform leverages a modern, decoupled technology stack organized into three domains:

#### 4.1 Frontend

- **React.js & Vite:** Provides a lightning-fast Single Page Application (SPA) experience.
- **Tailwind CSS & Framer Motion:** Used for building a responsive, animated, and accessible UI.
- **Lucide React & React Markdown:** Renders icons and structures LLM text outputs into readable formats.

#### 4.2 Backend

- **Node.js & Express.js:** Handles business logic, JWT authentication, and acts as a proxy gateway.
- **FastAPI (Python):** Serves the computationally heavy AI routing, transcription, and LangGraph execution due to Python's dominance in the AI ecosystem.
- **MongoDB:** NoSQL database for managing user profiles and conversation histories.

#### 4.3 AI / Data Pipeline

- **LangGraph:** Orchestrates the state-based decision-making loop of the Jagruk Brain.
- **Qdrant:** High-performance vector database storing the embedded BIS knowledge.
- **Sentence Transformers (all-MiniLM-L6-v2):** Generates dense vector embeddings for semantic similarity.
- **Groq (GPT-OSS-120B):** Provides ultra-low latency inference for query understanding and response generation.
- **Web Scraping / Playwright:** Powers the initial automated data ingestion pipeline.

## CHAPTER 5 — BIS DATA COLLECTION AND KNOWLEDGE PREPARATION

### BIS Data Collection & Document Extraction Pipeline

Automated workflow architecture for standard document harvesting and processing

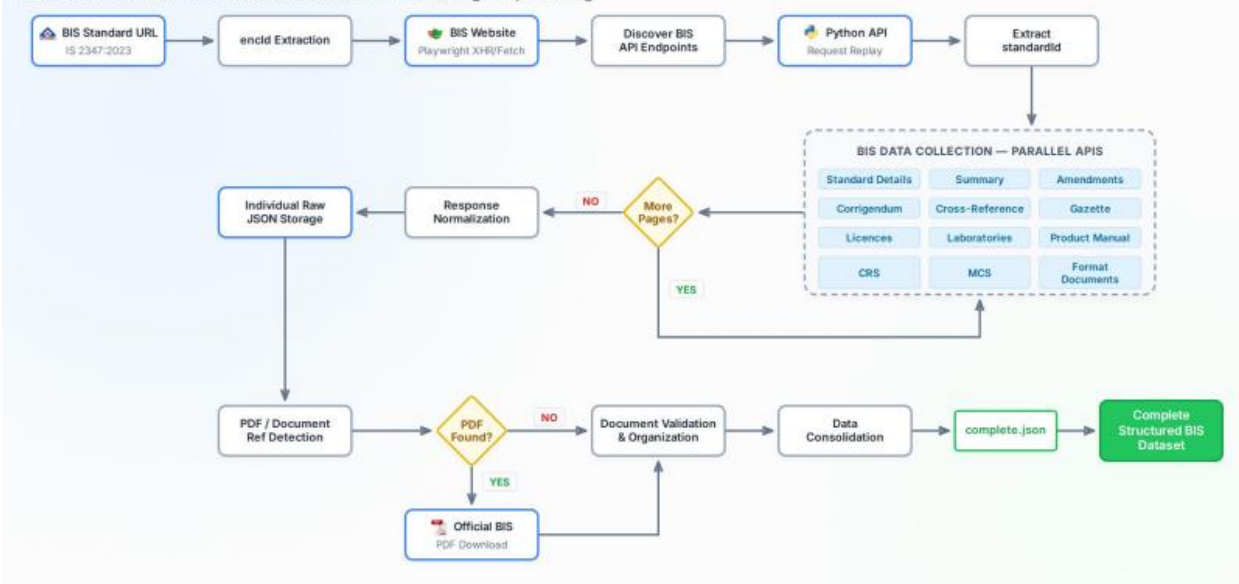


Figure 5.1: BIS Data Collection & Document Extraction Pipeline (Referencing Project Data Architecture)

The foundation of the RAG system is a high-quality dataset. The system employs an automated extraction pipeline:

### 5.1 to 5.7 Ingestion and Discovery

The pipeline begins at the BIS Standard URL. Using **Playwright**, the system simulates XHR/Fetch requests to discover hidden BIS API endpoints. Python scripts then replay these requests to extract Standard IDs dynamically.

### 5.8 Parallel Data Extraction

The pipeline pulls data across multiple categories simultaneously:

- Standard Details & Summaries
- Amendments & Corrigendum
- Cross-References & Gazettes
- Product Manuals & Licenses

### 5.12 to 5.17 Processing and Consolidation

PDF references are detected and downloaded. The raw data is validated, organized, and consolidated into a master complete.json file. This structured dataset is critical for minimizing noise before chunking and embedding, ensuring the AI has pristine context to draw from.

## 6.1 What is RAG?

Retrieval-Augmented Generation connects an LLM to external data sources. Instead of relying on the model's internal weights (which may be outdated or hallucinated), Jagruk Swadesh retrieves factual BIS documents first.

## 6.4 to 6.8 Document Preparation & Embedding

The complete.json data is parsed and split into manageable chunks. Each chunk is passed through the **all-MiniLM-L6-v2** model to generate a dense numerical vector (embedding).

## 6.9 Qdrant Vector Database

These vectors are stored in **Qdrant**. When a user asks a question, their query is also embedded. Qdrant performs a Cosine Similarity search to find the nearest vectors—representing the most relevant BIS clauses.

## 6.17 Evidence-Grounded Generation

The retrieved chunks are injected into the Groq LLM prompt as strict context. The system is instructed to *only* answer based on this context, effectively eliminating hallucination.

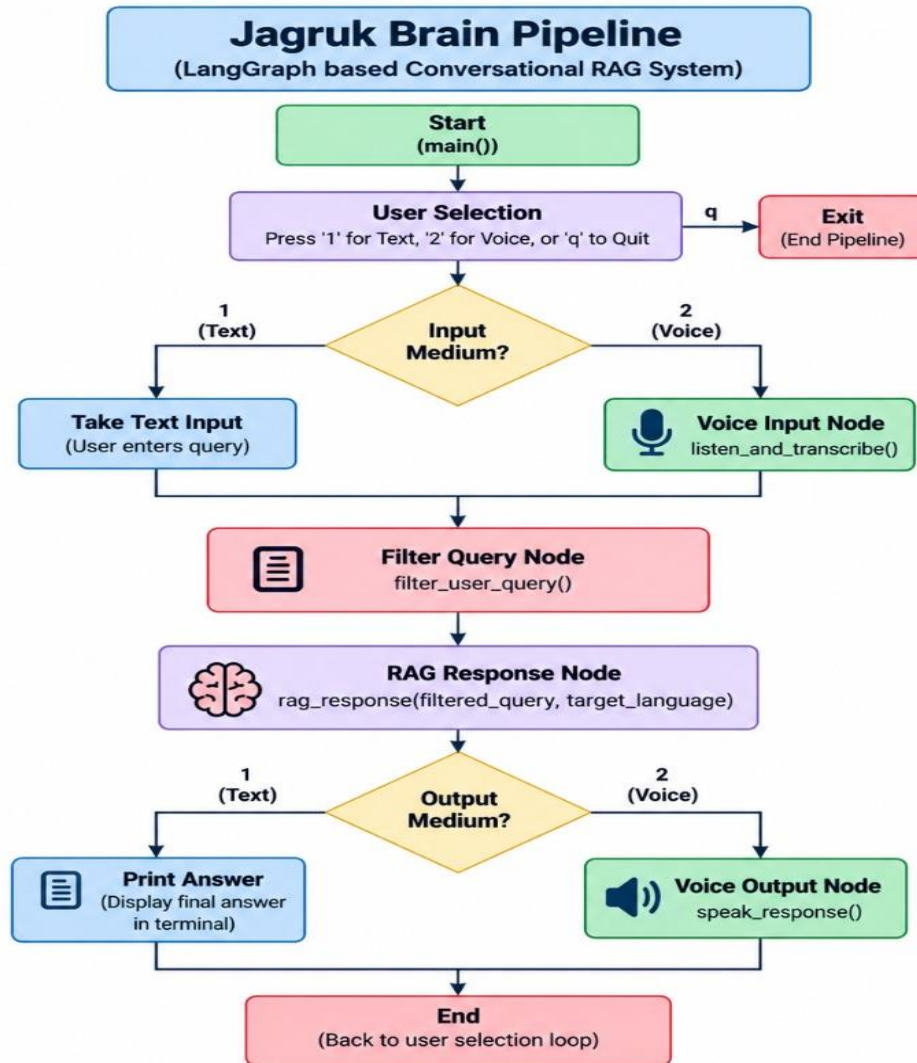


Figure 7.1: Jagruk Brain Pipeline

## 7.1 Pipeline Overview

The "Jagruk Brain" utilizes a state-based pipeline (implemented via LangGraph concepts) to route data efficiently.

## 7.4 Text vs 7.5 Voice Input Paths

The system decides the processing path based on the input medium.

- **Voice Input Node:** Captures audio and sends it to the Whisper model for Speech-to-Text (STT) conversion.
- **Text Input:** Bypasses STT and moves directly to processing.

## 7.10 Filter Query Node



Before searching the vector database, the query undergoes preprocessing: identifying the language, extracting the core intent, and identifying product names.

### **7.17 RAG Response Node**

This node queries Qdrant and passes the context to the LLM for reasoning.

### **7.20 Output Medium Decision**

If the user initiated the query via voice, the text response is routed to the **Voice Output Node** (Edge TTS) to generate spoken audio, closing the conversational loop seamlessly.

## **8.1 Natural Language Understanding**

The system intelligently bridges the gap between colloquial queries (e.g., "rules for ceiling fans") and official nomenclature ("IS 374:2019 Electric ceiling type fans").

## **8.9 Evidence-Grounded Response Generation**

Once the formal search terms retrieve the context, the Groq-powered LLM acts as an evaluator. It reads the retrieved text and formulates a response in the user's requested language, ensuring technical parameters (like safety thresholds) are cited exactly as they appear in the source.

## **CHAPTER 9 — VOICE AND MULTILINGUAL INTERACTION**

### **9.1 Voice Input and Transmission**

Users can speak directly into the browser UI. The frontend records an audio blob and POSTs it to the backend.

### **9.5 Whisper**

The system utilizes Whisper for highly accurate, multilingual Speech-to-Text transcription. Whisper handles regional accents effectively.

### **9.8 Text-to-Speech**

Responses generated by the AI are converted back to audio using TTS libraries, ensuring accessibility for users who prefer auditory learning or face literacy barriers.

## **CHAPTER 10 — FRONTEND APPLICATION**



*Figure 10.1: Jagruk Swadesh Frontend User Workflow*

The frontend provides an intuitive, accessible journey.

### 10.1 to 10.6 User Journey

- **Home:** Landing page explaining the value proposition.
- **Sign Up / Login:** Secure access to personalized features.
- **Explore:** Categorized search for specific standards.
- **Ask AI:** The core chat interface handling markdown text and voice interactions.

### 10.7 React Architecture

Built with Vite for rapid module replacement, the UI uses functional components and React hooks for state management. **Tailwind CSS** ensures the application is responsive across mobile and desktop devices.

## CHAPTER 11 — BACKEND ARCHITECTURE

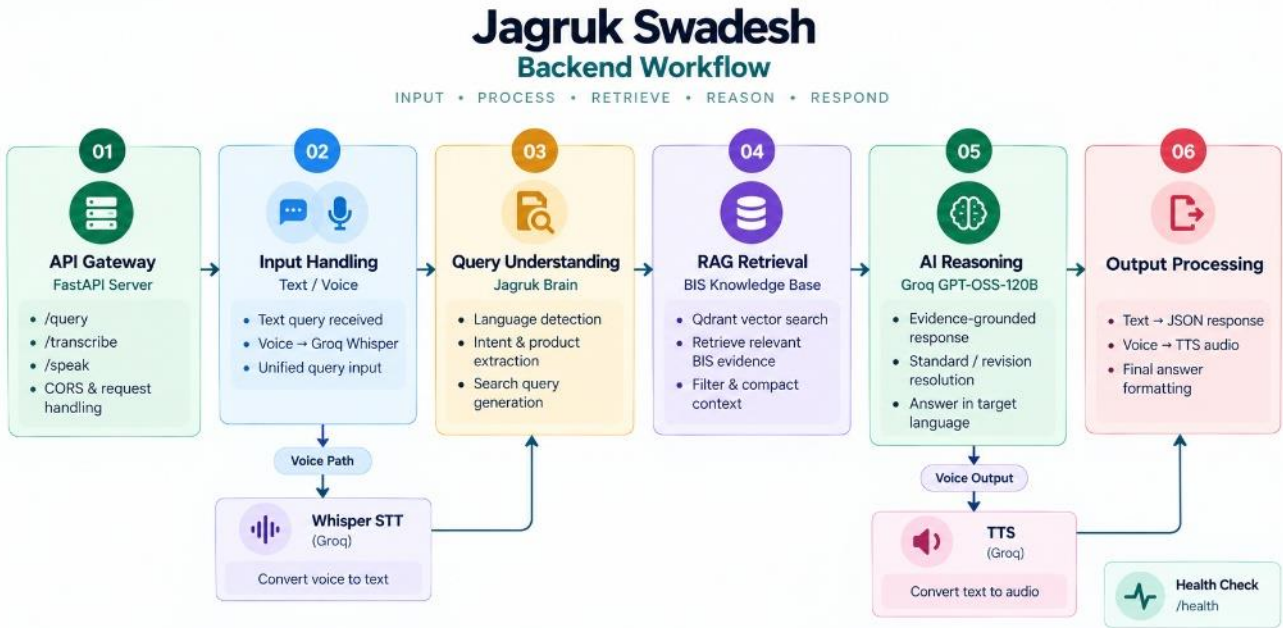


Figure 11.1: Jagruk Swadesh Backend Workflow

## 11.1 API Gateway

The backend bridges the user interface with the AI processing layer. An Express.js server handles standard HTTP requests, authenticates users, and proxies complex AI tasks to a dedicated FastAPI service.

### 11.8 to 11.10 Endpoints

- **Input Handling:** Accepts raw text or audio.
- **Query Understanding:** Routes to Jagruk Brain.
- **RAG Retrieval & AI Reasoning:** Interfaces with Qdrant and Groq.
- **Output Processing:** Formats JSON responses or audio streams.

## CHAPTER 12 — API DESIGN AND DATA FLOW

Table 12.1: API Endpoints

| Endpoint         | Method | Purpose           | Payload / Processing            |
|------------------|--------|-------------------|---------------------------------|
| /api/auth/signup | POST   | User registration | Hashes password, saves to Mongo |

| Endpoint        | Method | Purpose          | Payload / Processing                  |
|-----------------|--------|------------------|---------------------------------------|
| /api/auth/login | POST   | User login       | Validates credentials, returns JWT    |
| /query          | POST   | Text AI Query    | Receives text, triggers RAG           |
| /transcribe     | POST   | Audio processing | Receives audio file, uses Whisper STT |
| /speak          | POST   | Audio generation | Receives text, returns TTS stream     |
| /health         | GET    | System status    | Checks service uptime                 |

## CHAPTER 13 — DATABASE AND DATA MANAGEMENT

### 13.1 Application Database

**MongoDB** (via Mongoose) stores user credentials securely, managing application state, sessions, and configuration metadata.

### 13.10 Vector Database

**Qdrant** is strictly isolated for AI vector search. It stores the multidimensional embeddings of the BIS text chunks, allowing rapid semantic similarity calculations at scale.

## CHAPTER 14 — SECURITY

### 14.1 Authentication & Protection

- **JWT:** Secures API routes and maintains stateless sessions.
- **Password Hashing:** Passwords are encrypted before database insertion.
- **Environment Variables:** Sensitive keys (Groq API, Qdrant credentials, JWT secrets) are kept out of source code.
- **CORS:** Explicitly configured to prevent unauthorized cross-origin requests.

## CHAPTER 15 — TESTING AND VALIDATION

### Table 15.1: Recommended Test Cases

| Test ID | Module   | Test Scenario              | Expected Result                          |
|---------|----------|----------------------------|------------------------------------------|
| TC01    | Auth     | Login with valid JWT       | 200 OK, User Profile returned            |
| TC02    | AI Brain | Query generic greeting     | Direct LLM response (No RAG)             |
| TC03    | AI Brain | Query specific IS standard | Retrieve exact context, factual response |
| TC04    | Voice    | Upload clear audio query   | Accurate text transcription              |

## CHAPTER 16 — DEPLOYMENT AND SETUP

### 16.1 Development Environment



Requires Node.js for the frontend/gateway and Python 3.10+ for the FastAPI service.

### **16.13 Recommended Production Architecture**

- **Frontend:** Deployed to Vercel for global CDN edge caching.
- **Backend Services:** Hosted on Railway for seamless Dockerized deployment.
- **Databases:** MongoDB Atlas (Application) and Qdrant Cloud (Vector).

## **CHAPTER 17 — PERFORMANCE, SCALABILITY AND RELIABILITY**

The microservices design allows the FastAPI heavy-computation node to be scaled independently of the Express web server. Qdrant is inherently scalable for millions of

vectors, ensuring retrieval times remain in the low milliseconds regardless of dataset size.

## CHAPTER 18 — LIMITATIONS AND CHALLENGES

- **API Dependency:** Reliant on external uptime (Groq, Qdrant).

- **Speech Recognition:** Heavy background noise can degrade Whisper STT accuracy.
- **Data Freshness:** BIS documents update frequently; the vector DB requires periodic re-syncing to avoid serving deprecated standards.

## CHAPTER 19 — FUTURE SCOPE

- **Continuous Synchronization:** Automated webhooks to update the vector database whenever a new standard is gazetted.

- **More Indian Languages:** Expanding TTS and STT to fully support deep regional dialects.
- **Manufacturer Portals:** Dashboards for businesses to track their specific compliance journey based on AI recommendations.

## CHAPTER 20 — IMPACT AND BENEFITS

Jagruk Swadesh directly tackles the friction of regulatory compliance. By turning monolithic legal texts into conversational, accessible interactions, it reduces the

knowledge gap between policymakers and citizens, fundamentally aiding in the digital transformation of India's manufacturing and consumer safety sectors.

## **CHAPTER 21 — CONCLUSION**

Jagruk Swadesh successfully implements a state-of-the-art conversational RAG system tailored specifically for Indian regulatory compliance. By orchestrating a modern React frontend, a robust Node/FastAPI backend, and the LangGraph-powered "Jagruk Brain," the platform achieves accurate, fast, and secure information retrieval. Integrating voice capabilities and semantic search fundamentally transforms the user experience from

manual document hunting to seamless dialogue, proving the project's core thesis:  
*"Indian Standards Made Simple."*

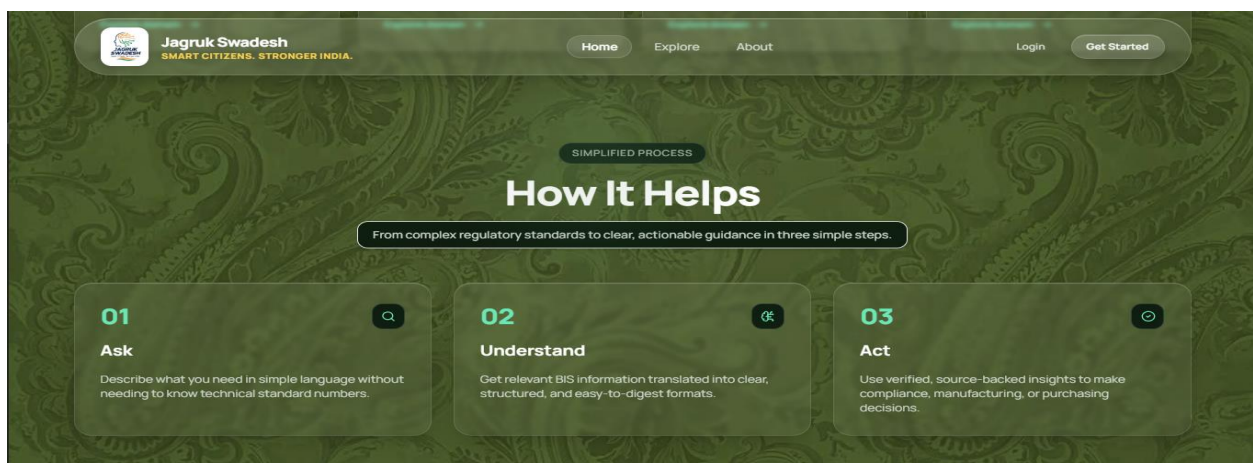
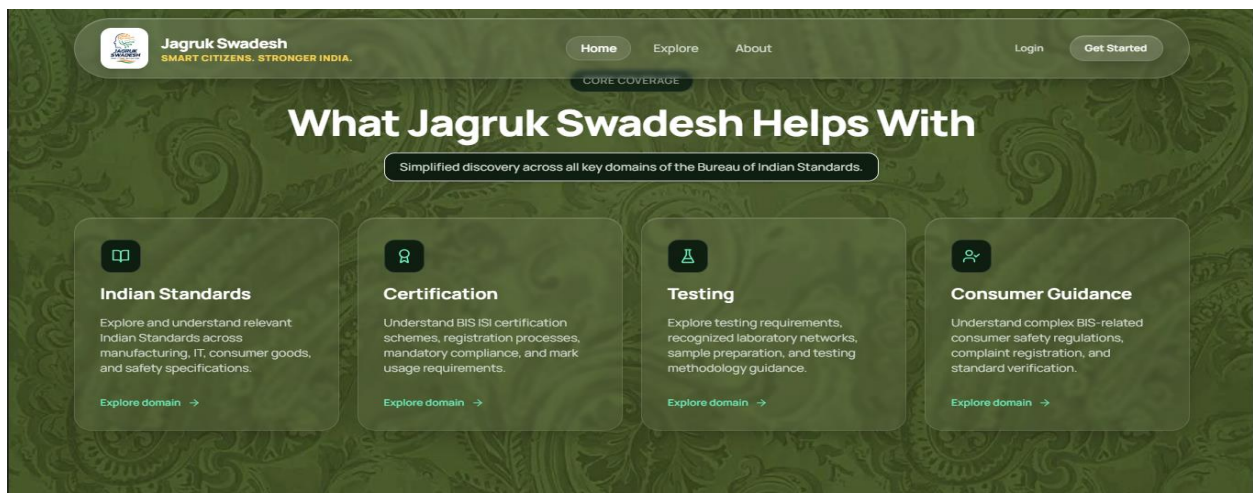
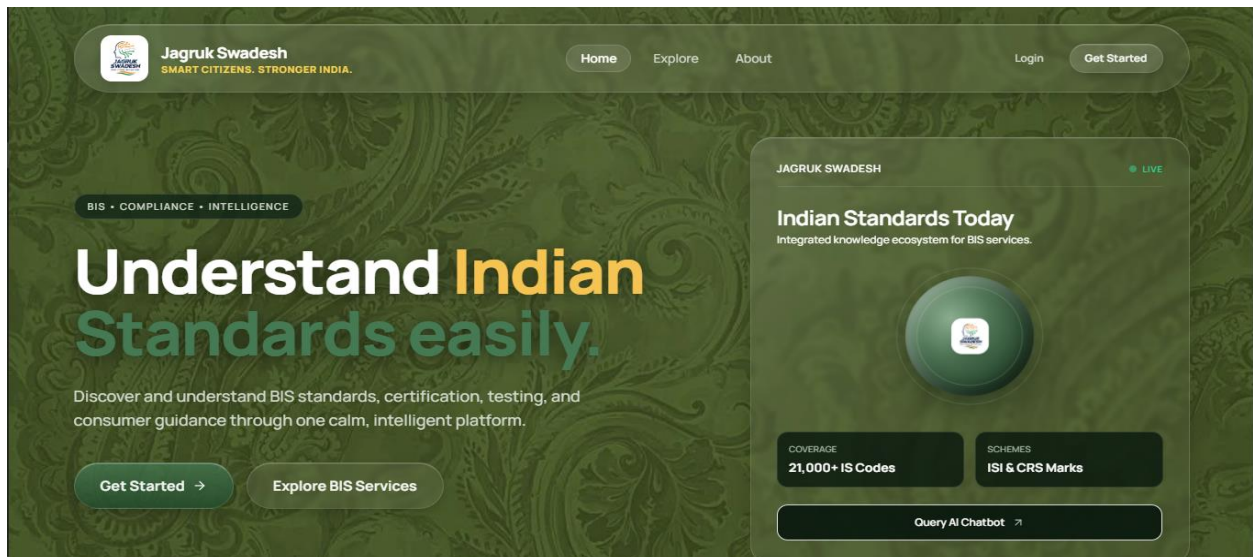
## **APPENDICES**

**APPENDIX A: Project Directory Structure** Contains isolated directories for /frontend, /backend, and /ai\_pipeline, enforcing separation of concerns.

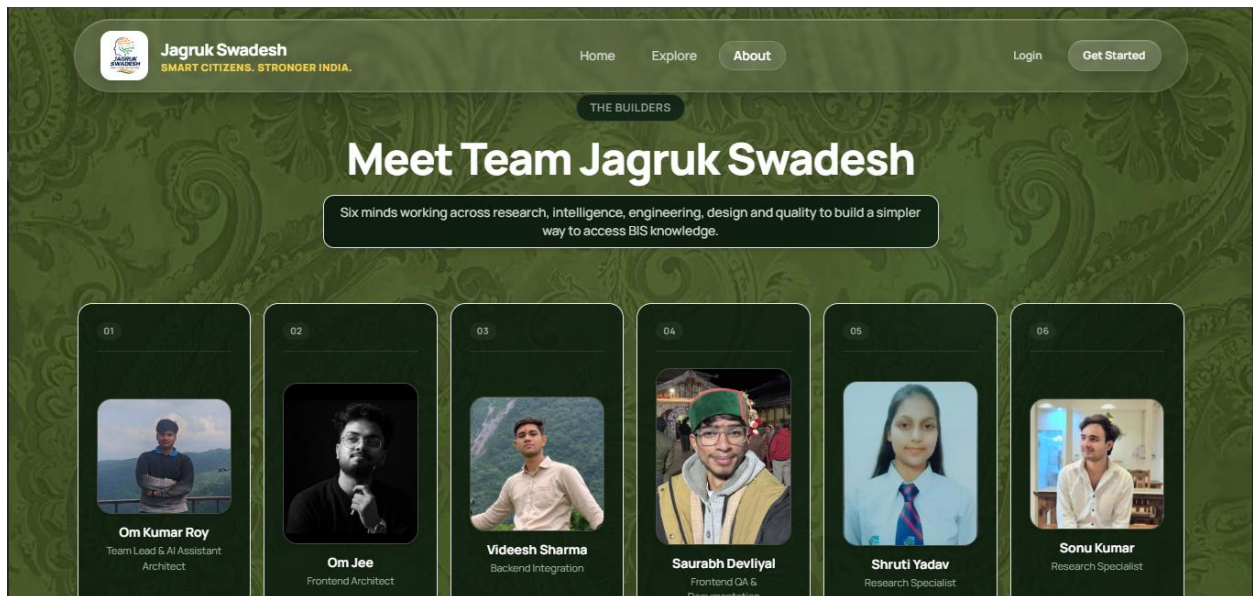
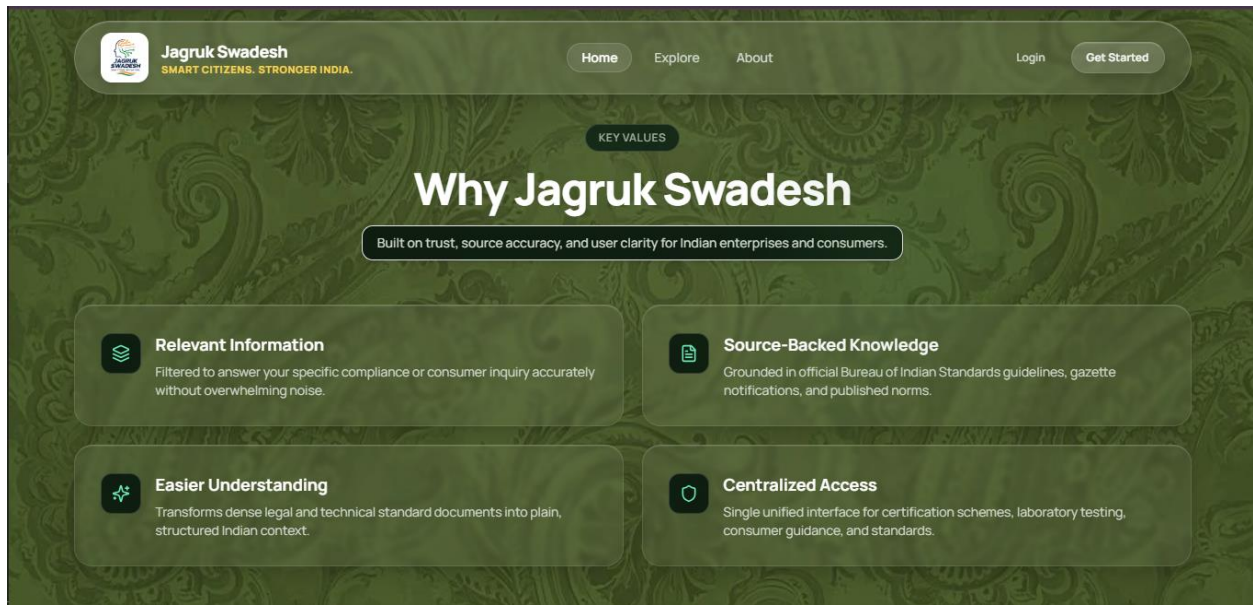
**APPENDIX B: Technology Stack** Comprehensive listing of package dependencies (React, Express, FastAPI, LangChain, Qdrant-client).

**APPENDIX C: Environment Variables** Required keys: MONGO\_URI, JWT\_SECRET, GROQ\_API\_KEY, QDRANT\_URL. (Actual values excluded for security).

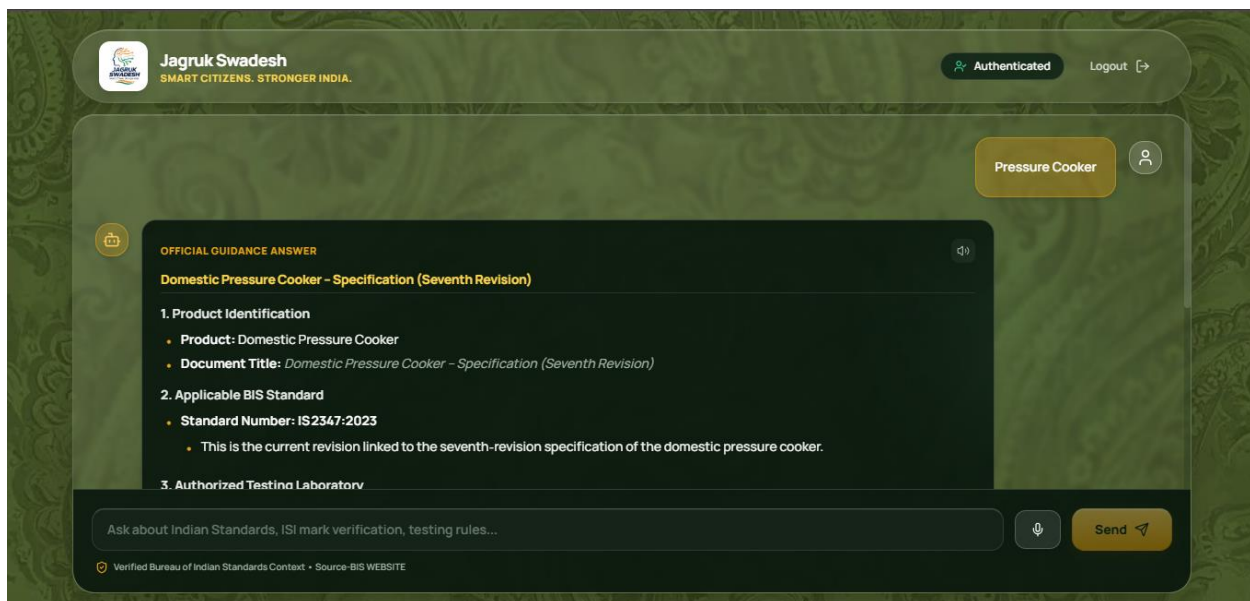
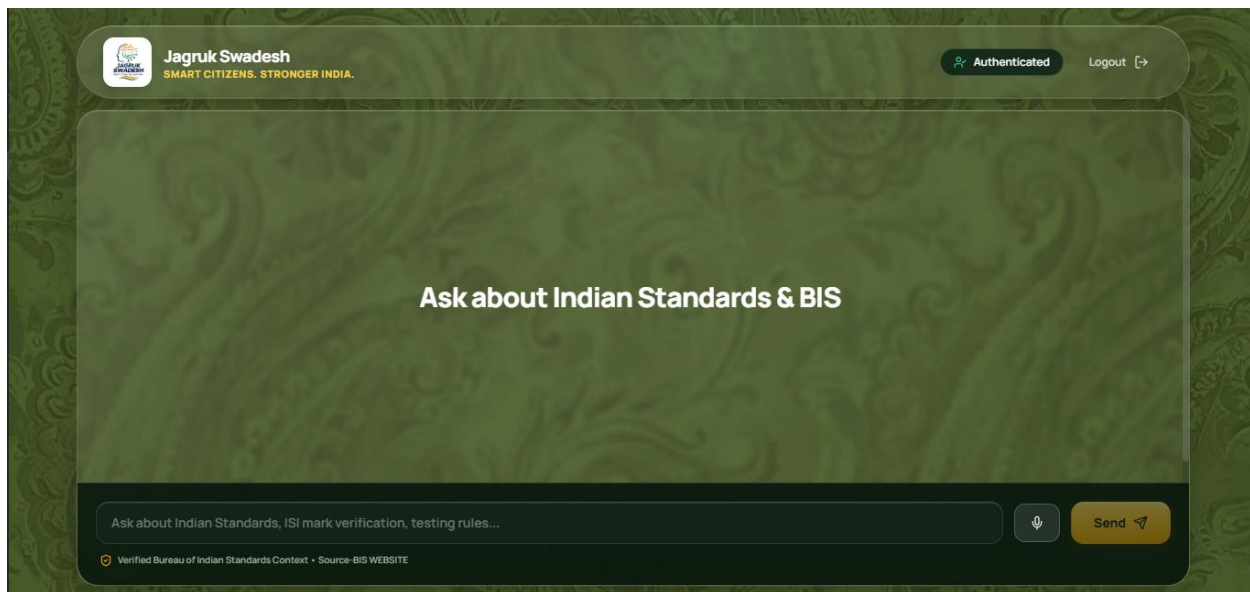
## **WEBSITE UI**







## CHATBOT UI AND RESPONSE



**Jagruk Swadesh**  
SMART CITIZENS. STRONGER INDIA.

Authenticated Logout

### 2. Applicable BIS Standard

- Standard Number: IS2347-2023
  - This is the current revision linked to the seventh-revision specification of the domestic pressure cooker.

### 3. Authorized Testing Laboratory

The BIS records list one laboratory approved for testing under IS2347:2023.

| # | LABORATORY NAME                                                | ADDRESS                                                             | STATE      | STANDARD COVERED |
|---|----------------------------------------------------------------|---------------------------------------------------------------------|------------|------------------|
| 1 | Scientific and Industrial Testing and Research Centre (SITARC) | 83 & 84 Avarampalayam Road, K.R. Puram Post, Coimbatore, Tamil Nadu | Tamil Nadu | IS 2347:2023     |

- Unique Laboratories Found: 1 (as per the deterministic laboratory summary).

### 4. Availability of Specification Details

The retrieved BIS evidence provides the product name, the applicable standard (IS2347:2023), and the authorized laboratory (SITARC). The detailed technical specifications of the seventh-revision domestic pressure cooker are not included in the available data.

If you need the full specification text, you may need to consult the official BIS publication of IS 2347:2023 or contact the listed

Send

Verified Bureau of Indian Standards Context • Source: BIS WEBSITE